Spring Boot

Spring Boot is an extension of Spring Framework that makes building backend applications:

-> Faster
-> Easier
-> Less configuration-heavy
-> Production ready

In simple words:

Spring Boot lets you create a complete backend application
with Spring in minutes instead of hours.

Spring Boot is NOT a new framework replacing Spring.

--> It USES Spring internally.

Why Backend Frameworks Exist (Life Before Spring)

Before frameworks like Spring, backend development in Java was mainly done using:

-> Servlets
-> JSP (Java Server Pages)
-> JDBC for database
-> Application servers like Tomcat, WebLogic

Typical flow:
Browser → Servlet → JDBC → Database

Sounds simple — but in reality it was messy.

Problems in early backend development:

A LOT of boilerplate code

Example in JDBC:

-> Open connection
-> Create statement
-> Execute query
-> Handle result set
-> Close connection
-> Handle exceptions

--> You had to write this again and again in every project.

Tight coupling (hard dependency)

Example:

```
MySQLDatabase db = new MySQLDatabase();
```

If tomorrow you want PostgreSQL → you change code everywhere.

This makes:

-> Code hard to maintain
-> Hard to test
-> Hard to scale

No proper structure

Each developer wrote code differently:

-> No standard architecture
-> No dependency management
-> No lifecycle management

Manual configuration hell

Everything was configured manually in:

-> XML files
-> web.xml
-> server configs

-> Small mistake = server won't start

Conclusion:

Backend without frameworks was:

-> Slow to develop
-> Hard to maintain
-> Error-prone
-> Not scalable

That's why backend frameworks were needed.

Problems with Servlets, JDBC, and XML Configuration

Servlets Problems

1. Too low-level

You had to manage:

Request parsing
Response writing
Thread handling
Session management

Everything manually.

2. No separation of concerns

Business logic + request handling mixed together.

Result:
-> Spaghetti code

JDBC Problems

1. Boilerplate code (already discussed)
2. Resource leaks

If you forget to close connection → memory leak.

3. Hard to switch databases

XML Configuration Problems

Spring early days used heavy XML.

Example:
```
<bean id="userService" class="com.app.UserService">
```

Large projects had:
1000+ lines of XML

Problems:

Hard to read
Hard to debug
Not developer friendly

What is Spring and Why It Became Popular

Spring was created to fix these exact problems.
Core Idea of Spring
-> Make Java backend simple, clean, and loosely coupled.

Spring introduced:

Dependency Injection (DI)

Instead of:
new MySQLDatabase()

Spring gives you object automatically.

This means:

- Easy to change implementations
- Easy testing
- Loose coupling

Inversion of Control (IoC)

Spring manages:

- Object creation
- Object lifecycle
- Dependency wiring

You focus only on business logic.

Spring JDBC & ORM support

It simplified DB work:

- Less boilerplate
- Better exception handling
- Integration with Hibernate/JPA

MVC architecture

Spring MVC separated:

- Controller
- Service
- Repository

Clean structure.

Why Spring become popular

- Solved boilerplate problem
- Loose coupling
- Clean architecture
- Huge ecosystem
- Enterprise-ready

What Problems Spring Had

Spring was powerful — but not easy.

Problem 1: Too much configuration

Old Spring projects required:

- XML files
- Bean definitions
- Web configs
- Dispatcher servlet setup

Starting a project took TOO MUCH time.

Problem 2: Hard setup

You had to:

- Download jars manually (early days)
- Configure server
- Setup web.xml
- Configure Spring context

For beginners → nightmare.

Problem 3: Version compatibility

One wrong dependency version and:

--> App won't start

Problem 4: Not developer friendly for fast development
In startups & microservices world:

People wanted:

- Quick setup
- Less config
- Production ready defaults

Spring was heavy.

Why Spring Boot Was Created

Spring Boot was created to:

-> Make Spring EASY
-> Remove configuration pain
-> Enable rapid development

Main goals of Spring Boot:

-> Zero (or minimal) configuration
-> Auto setup everything
-> Embedded servers
-> Opinionated defaults
-> Production ready features

Key Innovations of Spring Boot

1. Auto Configuration

Spring Boot automatically configures:

- Database
- Web server
- Security
- JSON handling

Based on:
- Dependencies you add

Example:

Add spring-boot-starter-web →

Spring Boot auto sets:

-> Tomcat
-> Spring MVC
-> Jackson JSON

No XML.

2. Starters (dependency bundles)

Instead of 10 libraries:

```
spring-boot-starter-web
```

It includes everything needed.

3. Embedded server

No need external Tomcat.

Just run:

```
java -jar app.jar
```

Boom server running.

Perfect for:

-> Microservices
-> Cloud
-> Docker

4. Production ready features

Spring Boot Actuator gives:

- Health checks
- Metrics
- Monitoring
- Logs

Important for modern systems.

-> Spring fixed backend complexity.
-> Spring Boot fixed Spring's complexity.

Spring Boot is one of the most widely used backend frameworks in enterprise and cloud-native applications.

If someone learns:

-> Java + Spring Boot

They are highly employable in:

-> Product companies
-> Service companies
-> Startups

How Much Java You Need for Spring Boot? (Non-negotiable)

OOP Concepts (VERY IMPORTANT)

You should understand:

- Class & Object
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction
- Interfaces

-> Spring heavily uses interfaces + implementations.

Basic Java syntax

- Variables
- Methods
- Loops
- Conditions

Packages & access modifiers

-> public, private, protected
-> package structure

Important for component scanning.

Exception handling

-> try-catch
-> custom exceptions

Spring uses exceptions everywhere.

Collections framework

Must know:

- List
- Map
- Set

Used in APIs, DB results, JSON mapping.

Spring Core Concepts

Lets try to Achieve Loose Coupling

Tightly Coupled System (problem)

Let's say we have:

-> A service that saves user data
-> It directly uses MySQL database

Problems:

UserService depends directly on MySQLDatabase

If tomorrow we switch to PostgreSQL → code change

Hard to test (can't mock easily)

Huge systems become unmanageable

We want:

UserService should NOT care:

-> which database
-> how it's created

It should just say:

--> "I need something that can save data"

So we introduce an INTERFACE (best practice)

Now we achieved:

Loose coupling
Easy change
Easy testing

BUT...

Still a problem...

WHO will create objects?

Somewhere you still need:

```
Database db = new MySQLDatabase();
UserService service = new UserService(db);
```

In big projects:

-> Hundreds of objects
-> Thousands of dependencies

This object creation code becomes a MESS

This is where Spring Core enters

Spring says:

"Give me responsibility of creating and connecting objects."

This leads to:

Inversion of Control (IoC)

Before:

Your code controls object creation

After:

Spring controls object creation

-> So control is inverted

Instead of:
`new MySQLDatabase()`

You say:

-> "Spring, you create it"

IoC Container (the brain)

Spring has a container called:

-> ApplicationContext

It:

Creates objects
Stores them
Injects them
Manages lifecycle

Think of it like:

-> A big box of ready-made objects

Beans

objects managed by Spring

Example:

```
@Component
class DBService {
}
```

Spring will:

Create DBService object
Store it in container
Manage it

Now DBService is a Bean

Not every object is bean.

ONLY those created by Spring are beans.

How Spring knows what to create (Annotations)

Spring scans your project and looks for special annotations.

Main ones:

@Component

Generic bean

```
@Component
class DBService {
}
```

Means:

-> "Spring, create this object"

@Service

For business logic layer

```
@Service
class UserService {
}
```

@Repository

For database layer

```
@Repository
class UserRepository {
}
```

Are they different internally?

NO (almost same)

BUT:

They add semantic meaning:

Annotation	Purpose
@Component	General
@Service	Business logic
@Repository	DB layer (extra exception handling)

Modern best practice:

-> Use proper one for clarity

Dependency Injection (DI)

Spring automatically provides required objects to a class.

You don't do:

```
new MySQLDatabase()
```

Spring does:

```
-> inject it
```

This achieves:

```
Loose coupling
Clean code
Easy testing
```

Why constructor injection is BEST (modern)

Because:

```
Forces required dependency
No null values
Immutable
Works best with testing frameworks
```

That's why Spring Boot encourages it.

How Spring chooses which implementation

If multiple implementations exist:

```
@Repository
class MySQLDatabase implements Database {}

@Repository
class PostgreSQLDatabase implements Database {}
```

Spring gets confused

Option 1 — @Primary

Example:

```
@Primary
@Repository
class MySQLDatabase implements Database {}
```

Now Spring says:

-> "Use this one by default"

When to use @Primary:

```
When ONE implementation is main/default
Others are rarely used
Simple projects
```

Downside:

* Hardcoded in code
* Cannot change per environment easily

another option is @Qualifier but not recommended

Option 2 - Spring Profiles

Example:

```
@Profile("dev")
@Repository
class MySQLDatabase implements Database {}

@Profile("prod")
@Repository
class PostgreSQLDatabase implements Database {}
```

Activate profile:

```
spring.profiles.active=dev
```

(or environment variable in cloud)

Benefits:

```
Clean separation per environment
No if-else logic
Very scalable
```

Option 3 - @ConditionalOnProperty (BEST)

It means:

"Create this bean ONLY if a config property has a specific value"

So Spring decides implementation based on:

```
application.properties
application.yml
environment variables
```

(no code change!)

application.properties

```
db.type=mysql
```

MySQL implementation

```
@Repository
@ConditionalOnProperty(name = "db.type", havingValue = "mysql")
class MySQLDatabase implements Database {
}
```

PostgreSQL implementation

```
@Repository
@ConditionalOnProperty(name = "db.type", havingValue = "postgres")
class PostgreSQLDatabase implements Database {
}
```

Spring will automatically:

```
-> Create ONLY the matching bean
-> Inject it
```

No conflict. No ambiguity.

@Profile is mainly used for environment-based configuration,
while @ConditionalOnProperty provides fine-grained,
property-driven bean creation. In modern Spring Boot applications,

conditional configuration is preferred for feature toggles
and provider selection,
while profiles are used for environment separation.

Bean Scope

By default:

-> ONE object per bean (Singleton)

Meaning:

```
UserService everywhere = same object
Database everywhere = same object
```

Why?

```
Memory efficient
Fast
Thread safe usually
```

Other scopes (rare but useful):

```
prototype → new every time
request → per HTTP request
session → per user session
```

Bean Lifecycle

Spring doesn't just create object instantly.

Flow:

```
-> Create object
-> Inject dependencies
-> Call init method
-> Bean is ready
-> On shutdown → destroy
```

Hooks:

```
@PostConstruct
public void start() {
    System.out.println("Bean ready");
}

@PreDestroy
public void stop() {
    System.out.println("Cleaning up");
}
```

Used for:

```
Opening resources
Closing connections
```

Configuration (why & how)

We don't hardcode values like:

- DB url
- ports
- credentials

Instead use config files.

application.properties:

```
db.type=mysql
server.port=8080
```

inject:

```
@Value("${db.type}")
String dbType;
```

Better modern way:

```
@ConfigurationProperties(prefix="db")
class DBConfig {
   private String type;
}
```

(plus @ConfigurationPropertiesScan in main class)

Much cleaner

Java Config Beans

Sometimes we need custom objects.

```
@Configuration
class AppConfig {

   @Bean
   public Database database() {
      return new MySQLDatabase();
   }
}
```

Now Spring manages it as bean.

Spring Boot Internals: SpringApplication.run() + Auto Configuration

In Spring Core we learned about:

- IoC container
- Dependency Injection
- Bean lifecycle
- Component scanning

But in Spring Boot we usually just write: `SpringApplication.run(Application.class, args);`

And magically:

- the application starts
- the web server starts
- REST APIs start working
- configuration happens automatically

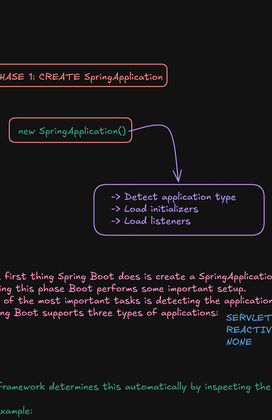
So the important question is:

What actually happens when this line executes?

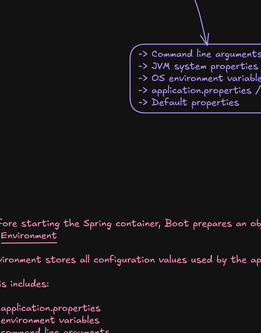
Specifically we will understand two major things:

-> What happens inside SpringApplication.run()
-> How Auto Configuration works internally

HIGH LEVEL STARTUP PIPELINE



PHASE 1: CREATE SpringApplication



-> The first thing Spring Boot does is create a SpringApplication object.
-> During this phase Boot performs some important setup.
-> One of the most important tasks is detecting the application type.
-> Spring Boot supports three types of applications:

SERVLET
REACTIVE
NONE

The framework determines this automatically by inspecting the classpath.

For example:

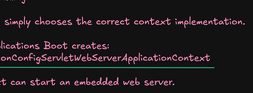
If classes from Spring MVC and the Servlet API are present,
Boot assumes this is a web application.

This mechanism is called:

Classpath-based configuration.

Spring Boot behaviour changes depending on the dependencies you include.

PHASE 2: PREPARE ENVIRONMENT



Before starting the Spring container, Boot prepares an object called:
Environment

Environment stores all configuration values used by the application.

This includes:

- application.properties
- environment variables
- command line arguments
- system properties

Spring Boot merges all these sources together.

But they follow a priority order.

1. Command line arguments
2. JVM system properties
3. OS environment variables
4. application.yml
5. Default values

Higher priority overrides lower priority.

This allows configuration to be overridden in production environments.

For example:
A property in application.yml can be overridden by a command line argument.

PHASE 3: CREATE APPLICATION CONTEXT

SERVLET → AnnotationConfigServletWebServerApplicationContext
REACTIVE → ReactiveApplicationContext
NONE → AnnotationConfigApplicationContext

SpringApplication now creates the ApplicationContext.

ApplicationContext is the IoC container provided by the Spring Framework.

It is responsible for:

- managing beans
- dependency injection
- lifecycle management
- event handling

Spring Boot simply chooses the correct context implementation.

For web applications Boot creates:
AnnotationConfigServletWebServerApplicationContext

This context can start an embedded web server.

AUTO CONFIGURATION

We now understand how Boot prepares the environment and context.

But another important question remains:

How does Boot automatically configure things like:

- DataSource
- Jackson
- Spring MVC
- Message converters

This is handled by Auto Configuration.

AUTO CONFIGURATION PIPELINE

```
@SpringBootApplication
        ↓
@EnableAutoConfiguration
        ↓
AutoConfigurationImportSelector
        ↓
Load AutoConfiguration.imports
        ↓
Apply Conditions
        ↓
Import Configuration Classes
```

The annotation @SpringBootApplication includes another annotation:
@EnableAutoConfiguration

This annotation enables the auto configuration mechanism.
Internally it imports a class called:

AutoConfigurationImportSelector

This class decides which configuration classes should be applied.

WHERE AUTO CONFIG CLASSES COME FROM

org.springframework.boot.autoconfigure.AutoConfiguration.imports

Spring Boot maintains a list of all possible auto configuration classes.

In Spring Boot 3 this list is stored in the file:
AutoConfiguration.imports

This file contains fully qualified class names of configuration classes.

Examples include:

- WebMvcAutoConfiguration
- DataSourceAutoConfiguration
- JacksonAutoConfiguration

Boot loads this list and then evaluates conditions.

CONDITIONAL FILTERING ENGINE

```
All Auto Configurations
        ↓
Check @ConditionalOnClass
        ↓
Check @ConditionalOnMissingBean
        ↓
Check @ConditionalOnProperty
        ↓
Valid Configurations
```

Each auto configuration class is protected by conditional annotations.

Some common ones include:

- @ConditionalOnClass
- @ConditionalOnMissingBean
- @ConditionalOnProperty
- @ConditionalOnWebApplication

These annotations ensure configuration only happens when appropriate.

For example:

@ConditionalOnClass(DataSource.class)

means the configuration will only apply if the DataSource class exists.

BACK-OFF STRATEGY

```
User Configuration
        >
Auto Configuration
```

Spring Boot follows a principle called Back-Off Strategy.
This means Boot only configures something if the user has not already defined it.

For example:

If the user creates a custom DataSource bean, Boot will not create the default one.
This behavior is controlled by the annotation:

@ConditionalOnMissingBean

FINAL STEP: CONTEXT REFRESH

```
Register configuration classes
        ↓
context.refresh()
        ↓
Bean creation
        ↓
Application Ready
```

After auto configuration classes are selected, they are registered with the Spring container.

Finally SpringApplication calls:
context.refresh()

This triggers the entire Spring container lifecycle:

- bean definitions processed
- dependencies resolved
- singleton beans created

After this step the application becomes fully operational.

FINAL RECAP

```
main()
  ↓
SpringApplication.run()
  ↓
Create SpringApplication
  ↓
Prepare Environment
  ↓
Create ApplicationContext
  ↓
Load AutoConfiguration.imports
  ↓
Apply Conditional Filtering
  ↓
Register Configurations
  ↓
context.refresh()
  ↓
Application Ready
```

Spring Boot startup is not magic.

It is a deterministic pipeline consisting of:

- environment preparation
- context creation
- conditional configuration selection
- container initialization

Understanding this pipeline makes the framework much easier to debug and customize.